Interview Question

Suppose A, B, C are uniform$(0,1)$, independent.

What is probability that $Ax^2 + 2Bx + C = 0$ has real roots?

The quadratic theorem is typically expressed in terms of $ax^2 + bx + c = 0$. In that case the roots are real if and only if $b^2 - ac \geq 0$

$2B = b$

$B = b/2$

Here, the extra "2" allows up to update this condition to $B^2 - AC \geq 0$.

$$P(B^2 - AC \geq 0) = P(B^2 \geq AC)$$

$$= \int_0^1 \int_0^1 P(B^2 \geq ac)\, da\, dc$$

using Law for Total Prob.

$$= \int_0^1 \int_0^1 P(B \geq \sqrt{ac}) \, da \, dc$$

$$= \int_0^1 \int_0^1 (1 - \sqrt{ac}) \, da \, dc$$

$$= 1 - \int_0^1 \int_0^1 \sqrt{ac} \, da \, dc$$

$$= 5/9$$

Columns 8 through 19: The Count Variables

The remaining column in the data set consist of trade and share counts of different types.

Note that some of the volume counts are reported in 1000's of shares, hence there are non-integer values among the counts.

Exercise: Execute the command below and comment on the results. It is especially important to focus on extreme or impossible values.

```
In [12]: fulldata.describe()
```

```
None
```

There are negative values for some counts, particularly for the "Hidden" columns

```
fulldata[fulldata["Hidden"]<0]
```

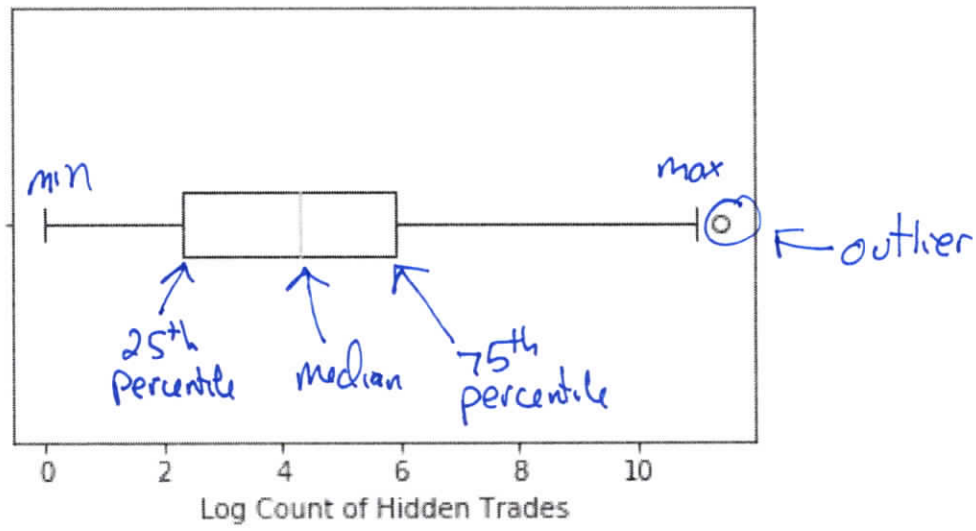
Note that cases where Hidden is negative corresponds to (typically) lightly-traded stocks

Negative values likely caused by taking differences of counts which each have some error in them

Of course, graphical tools are useful for understanding the properties of variables. This topic will be covered more fully later, but here we consider a classic approach: the **boxplot**:

```
In [13]: plt.figure(figsize=(5,3))
         plt.boxplot(np.log(fulldata['Hidden'])\
              .dropna(),labels=[""], vert=False)
         plt.xlabel("Log Count of Hidden Trades")
         plt.show()
```

```
/Users/chadschafer/anaconda3/lib/python3.7/site-packages/ipyke

/Users/chadschafer/anaconda3/lib/python3.7/site-packages/ipyke
```



The "box" of the plot extends from the 25th to the 75th percentile of the data, while the solid line in the middle of the box is at the median. The two "arms" extend to the minimum and maximum value, **except** that any value labelled an **outlier** is plotted separately.

Exercise: How is an outlier defined in this case?

The "arms" only extend to the largest value less than Q3 + whis * IQR, where
Q3 = third quartile.
whis = 1.5 by default
IQR = Q3 - Q1, rough measure of variability
Similar for lower end

Exercise: Why was the logarithm of the variable utilized? What issues did that create?

For many financial variables, we see a long upper tail

The log transforms serves to make the dist. more "symmetric", less skewed

Eg. in regression, skewed predictors tend to have highly influential cases

Exercise: Consider the outlier in the previous plot. Is this observation also extreme in the other variables? Can you find the source of this behavior?

This is the IPO for SNAP

fulldata.loc[fulldata["Hidden"].idxmax()]

Exercise: Parse the syntax, and comment on the output, of the following command.

```
In [14]: fulldata.iloc[:, 7:19].\
             apply(lambda x: x.idxmax())
```

```
None
```

The simple function "lambda" is applied to each column of fulldata

We are finding the row where each variable is maximized

Missing Data

Most data sets contain some amount of missing data, for many reasons.

As a first step, it is important to recognize how your data set represents missing values, so that they are read in properly.

In a `.csv` file such as that we are using here, it is common to simply have blank values to indiciate missingness, i.e., there are consecutive commas without a value between.

The `read_csv()` function has an argument `na_values` which allows the user to specify strings that should be considered as missing values. By default, the following are converted into `NaN`: `, #N/A, #N/A N/A, #NA, -1.#IND, -1.#QNAN, -NaN, -nan, 1.#IND, 1.#QNAN, N/A, NA, NULL, NaN, n/a, nan, null`. **Be careful:** It is not unusual for data sets to use numbers such as `-999` to indicate a missing value.

Pandas uses `NaN` to represent missing values.

Exercise: Execute the following commands and discuss the results

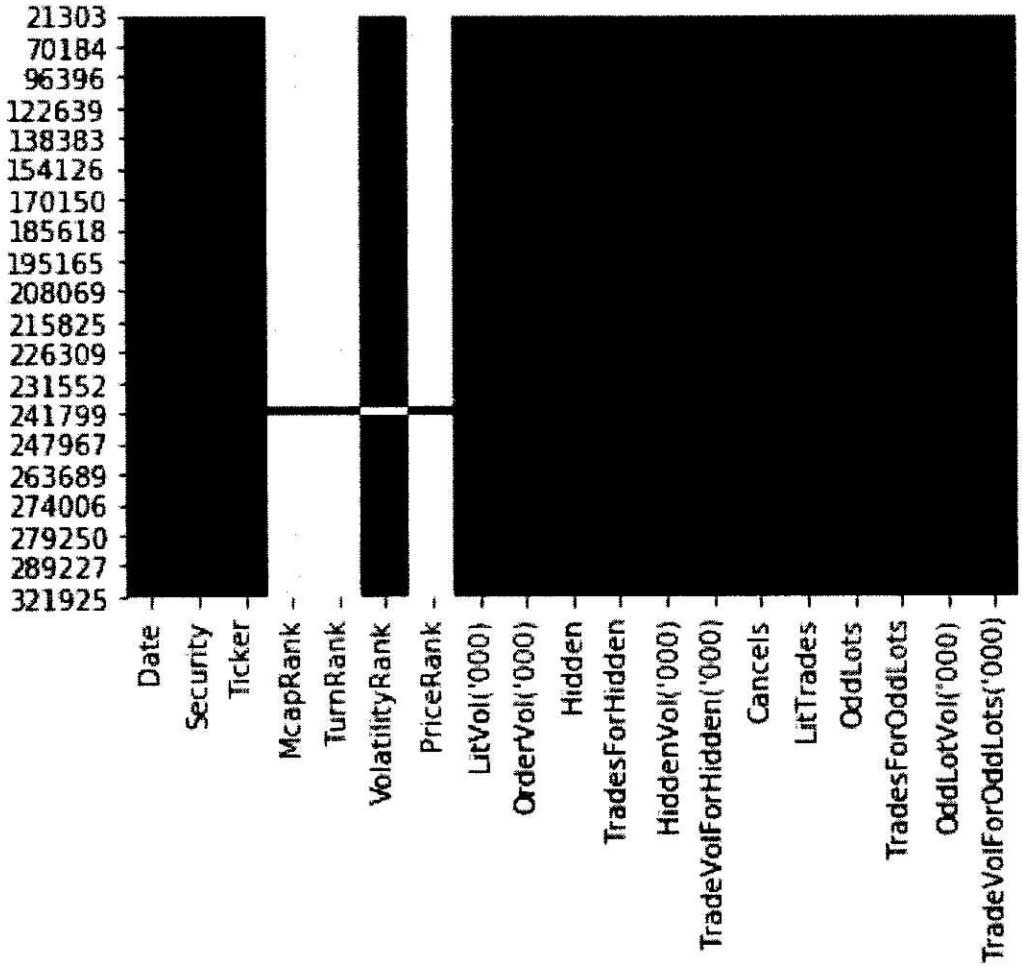
```
In [15]: np.where(fulldata.isnull())
         len(np.unique(np.where(fulldata.isnull())[0]))

Out[15]: 77
```

(1) returns all (row, col) that are missing

(2) How many rows have at least one missing value? 77

The package `seaborn` has a function `heatmap()` that can be useful for inspecting any patterns in the missingness.

```
In [16]: import seaborn as sns
         sns.heatmap(fulldata.iloc[np.unique\
             (np.where(fulldata.isnull())[0]),:]\
             .isnull(),cbar=False)
         None
```



The `dropna` function will remove any row from the data frame with at least one `NaN` value.

```
In [17]: fulldatacomplete = fulldata.dropna()
         len(np.unique(np.where(fulldatacomplete.\
                 isnull())[0]))
```

```
Out[17]: 0
```

Working only with such data is referred to as **complete case analysis**.

Exercise: Discuss potential drawbacks to complete case analyses.

The complete cases could differ from those with missing values in a systematic way

Could have a biased "training sample," not representative of the population

Exercise: Consider the output of the following and comment on any features of the incomplete cases that may be releveant.

```
In [18]: fulldata.iloc[np.unique(np.where(fulldata.\
             isnull())[0]),:]
```

```
None
```

Most rows with missing values correspond to stocks with light/no trading

Part 2: Basic Visualization

This Part covers basic tools for visualizing data in Python.

Visualization techniques are critical to discovering and understanding the relationships between variables.

```
In [1]: import numpy as np
        import pandas as pd
        import matplotlib.pyplot as plt
```

Example Data Set

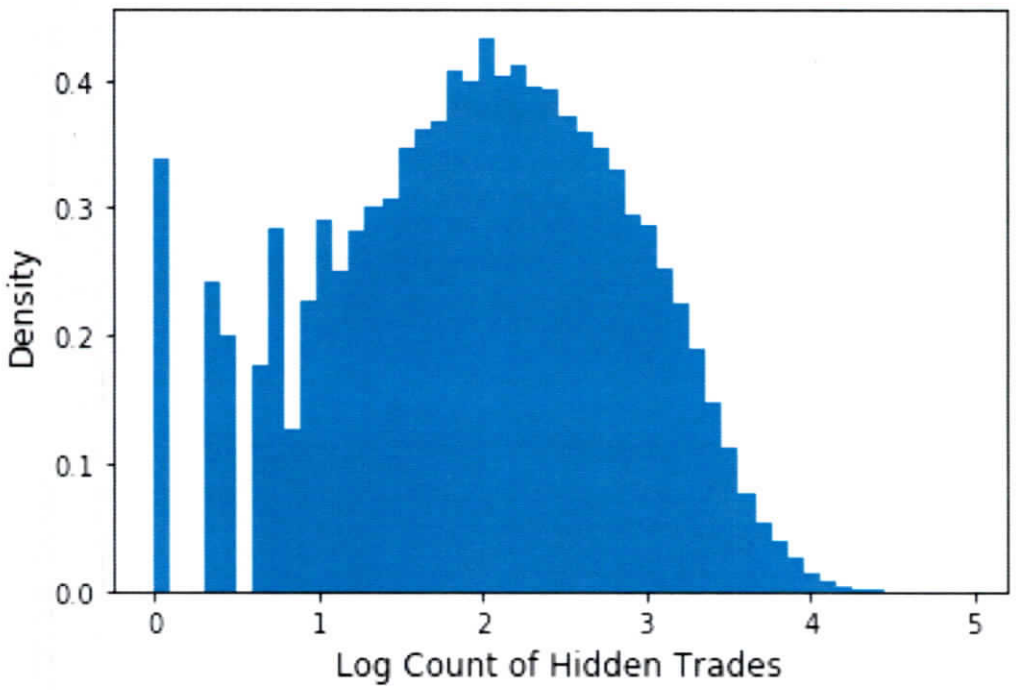
First, we will revisit the data set we considered in the previous Part, and the steps needed to properly format the data:

```
In [2]: fulldata =\
            pd.read_csv("q1_2017_all.csv", sep=",")
        fulldata['Date'] =\
            pd.to_datetime(fulldata['Date'],\
             format='%Y%m%d')
        fulldata =\
            fulldata.drop_duplicates(subset=\
             ('Date','Ticker'), keep='first')
        fulldata['McapRank'] =\
            fulldata['McapRank'].astype('category').\
             cat.as_ordered()
        fulldata['TurnRank'] =\
            fulldata['TurnRank'].astype('category').\
             cat.as_ordered()
        fulldata['VolatilityRank'] =\
            fulldata['VolatilityRank'].astype('category').\
             cat.as_ordered()
        fulldata['PriceRank'] =\
            fulldata['PriceRank'].astype('category').\
             cat.as_ordered()
```

Basic Plots

We saw the use of boxplot in a previous Part. A similar view of the shape of a distribution can be obtained using a **histogram**, formed using the `matplotlib` function `hist()`:

```
In [3]: fig = plt.figure(figsize=[6,4])
        axs = fig.subplots(1,1)
        axs.hist(np.log10(fulldata['Hidden']\
            [fulldata['Hidden'] > 0]),bins=50,density=True)
        axs.set_ylabel("Density", size=12)
        axs.set_xlabel("Log Count of Hidden Trades",size=12)
        plt.show()
```



Exercise: Parse the syntax of the code above to understand some basic aspects of `matplotlib`.

plt.fig() returns the basic "canvas" for the plot. Can set figure size, resolution, background color, ...

plt.subplots() divides the figure into the specified grid of plots

e.g. subplots(2, 2)

(2 → rows, 2 → cols.)

plt.show() displays the plot

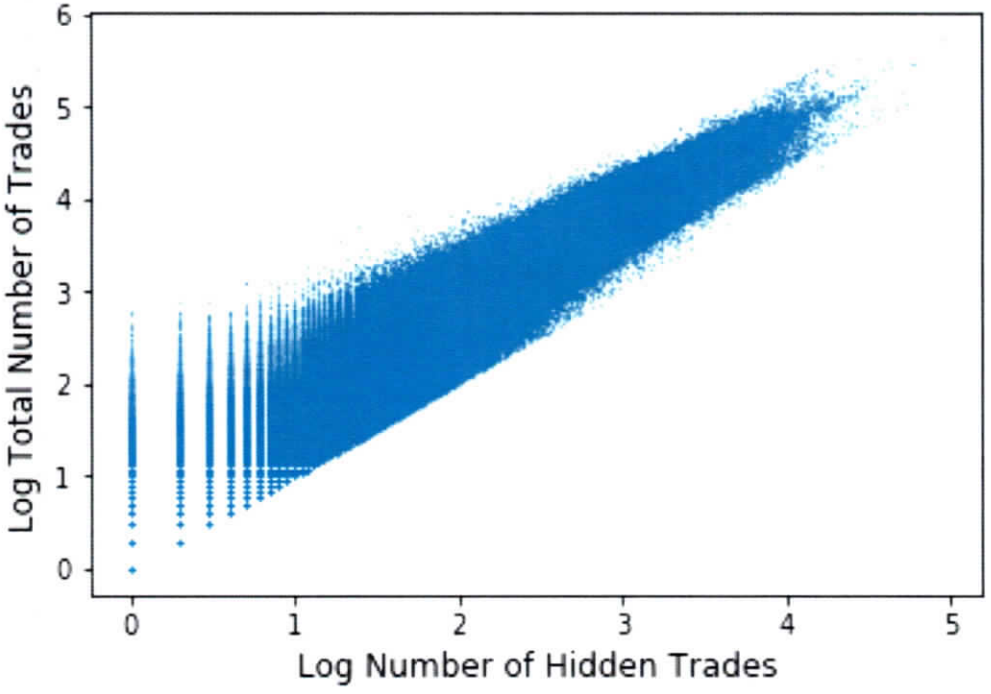
Exercise: How could this plot be improved?

Consider adjusting the number of bins.
Count is ~~very~~ large.

Make sure fonts readable. (adjust size)

Scatter plots are ubiquitous, as well. They are created simply in Python using the `plot()` method.

```
In [4]: with np.errstate(divide='ignore', invalid='ignore'):
            fig, axs = plt.subplots(1, 1, figsize=[6,4])
            axs.plot(np.log10(fulldata['Hidden']),
                 np.log10(fulldata['TradesForHidden']),
                 marker='.', ms=0.5, linestyle="none")
            axs.set_xlabel("Log Number of Hidden Trades",
                size=12)
            axs.set_ylabel("Log Total Number of Trades",
                size=12)
            plt.show()
```



Exercise: Comment on the Python syntax in the previous example.

"ms" controls the "marker size"

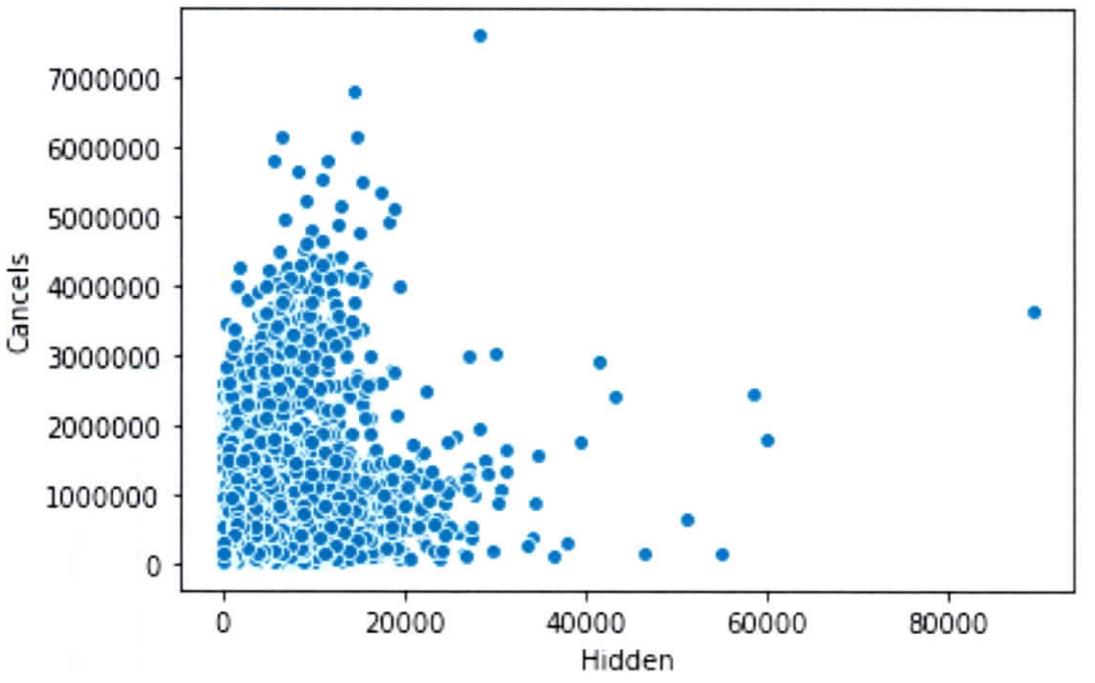
np.errstate controls how errors are handled

The Python package `Seaborn` allows for more sophisticated visualization. This package builds on `matplotlib` in useful ways. See https://seaborn.pydata.org/ for more detailed information and documentation.

```
In [5]: import seaborn as sns
```

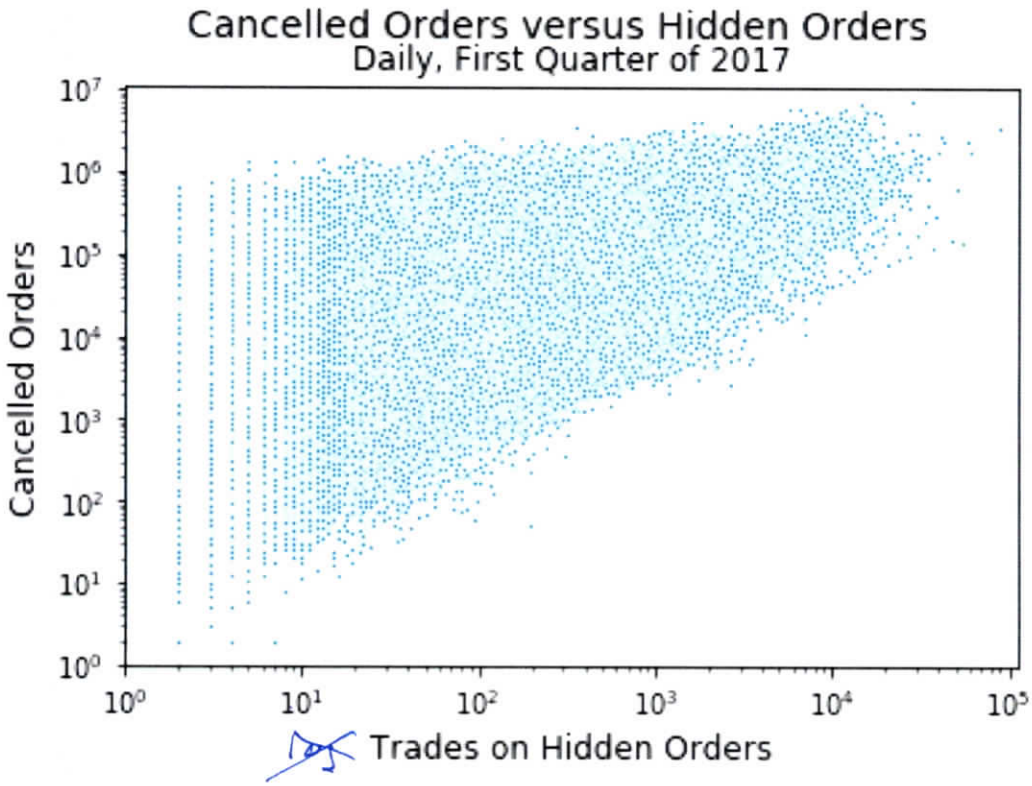
Let's start with a simple scatter plot, constucted via `scatterplot`:

```
In [6]: plt.figure(figsize=[6,4])
        ax = sns.scatterplot(x="Hidden", y="Cancels",
                data=fulldata)
```



Let's make an improved version.

```
In [7]: plt.figure(figsize=[6,4])
        ax = sns.scatterplot(x="Hidden", y="Cancels",
            data=fulldata, s=5)
        ax.set(xscale="log", yscale="log",
               xlim=(1, None), ylim=(1, None))
        plt.suptitle("Cancelled Orders versus Hidden Orders",
               size=14)
        plt.title("Daily, First Quarter of 2017")
        plt.xlabel("Trades on Hidden Orders", size=12)
        plt.ylabel("Cancelled Orders", size=12)
        plt.show()
```



Exercise: Comment on the improvements made here.

changed the marker sizes
put axes on log scale
improved titles and axis labels

The `hue` argument can be changed to vary with the levels of one of the categorical variables.

```
In [8]: plt.figure(figsize=[6,3.75])
        ax = sns.scatterplot(x="Hidden", y="Cancels",
            data=fulldata, s=5, hue="Security")
        ax.set(xscale="log", yscale="log",
               xlim=(1, None), ylim=(1, None))
        plt.suptitle("Cancelled Orders versus Hidden Orders",
               size=14)
        plt.title("Daily, First Quarter of 2017", size=10)
        plt.xlabel("Trades on Hidden Orders", size=12)
        plt.ylabel("Cancelled Orders", size=12)
        plt.show()
```